

Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №7

# ШАБЛОН «MEDIATOR», «FACADE», «BRIDGE», «TEMPLATE METHOD»

**Виконав**
студент групи ІА–22:
Артамонов Дмитро Вікторович
**Перевірив**:
Мягкий Михайло Юрійович

Київ 2024

**Зміст**

Хід роботи.....3
Короткі теоретичні відомості .....3
Реалізувати не менше 3-х класів відповідно до обраної теми .....5
Реалізувати один з розглянутих шаблонів за обраною темою .....6
Тестування патерну.....10
Висновок.....13

**Тема**: шаблони «MEDIATOR», «FACADE», «BRIDGE», «TEMPLATE METHOD».

**Мета**: ознайомитися з шаблонами проектування «MEDIATOR», «FACADE», «BRIDGE», «TEMPLATE METHOD» та набути практичних навичок їх застосування. Реалізувати частину функціоналу програми за допомогою одного з розглянутих шаблонів для досягнення конкретних функціональних можливостей та забезпечення ефективної взаємодії між класами.

## Хід роботи

**..15 E-mail клієнт (singleton, builder, decorator, template method, interpreter, SOA)**

Поштовий клієнт повинен нагадувати функціонал поштових програм Mozilla Thunderbird, The Bat і т.д. Він повинен сприймати і коректно обробляти

---

pop3/smtp/imap протоколи, мати функції автонастройки основних поштових провайдерів для України (gmail, ukr.net, i.ua), розділяти повідомлення на папки/категорії/важливість, зберігати чернетки незавершених повідомлень, прикріплювати і обробляти прикріплені файли.

## Короткі теоретичні відомості

### Шаблон «Mediator» (Посередник)

Основна ідея: координує взаємодію між компонентами через центральний об'єкт (посередника), що дозволяє зменшити залежність між класами та полегшити їхнє тестування і повторне використання.

Тип: поведінковий шаблон.

Елементи:

- Посередник (Mediator): інтерфейс для комунікації між компонентами.
- Конкретний посередник (Concrete Mediator): реалізує логіку взаємодії між об'єктами.
- Компоненти (Colleagues): об'єкти, що комунікують через посередника.

Переваги: спрощує підтримку коду, зменшує залежність між компонентами.

Недоліки: може стати центральною точкою перевантаження в складних системах.

**Шаблон «Facade» (Фасад)**

Основна ідея: приховує складність підсистеми за єдиним інтерфейсом, надаючи зручний доступ до її функціоналу.

Тип: структурний шаблон.

Елементи:

- Фасад (Facade): єдиний інтерфейс для клієнтів.
- Підсистема (Subsystem): складні внутрішні компоненти, що виконують основну роботу.

Переваги: зменшує зв’язність між клієнтом і системою, покращує зрозумілість і простоту використання.

Недоліки: не вирішує проблеми високої складності підсистеми.

**Шаблон «Bridge» (Міст)**

Основна ідея: розділяє абстракцію та її реалізацію, дозволяючи змінювати їх незалежно.

Тип: структурний шаблон.

Елементи:

- Абстракція (Abstraction): базовий інтерфейс для клієнтів.
- Розширена абстракція (Refined Abstraction): специфічні реалізації інтерфейсу.
- Реалізація (Implementation): інтерфейс для реалізацій.

- Конкретна реалізація (Concrete Implementation): конкретні деталі реалізації.

Переваги: спрощує розширення системи, дозволяє змінювати платформу без впливу на клієнта.

Недоліки: додає складності через додаткові класи.

**Шаблон «Template Method» (Шаблонний метод)**

Основна ідея: задає кроки алгоритму в базовому класі, а деталізацію залишає підкласам.

Тип: поведінковий шаблон.

Елементи:

- Базовий клас: містить скелет алгоритму.
- Підкласи: визначають реалізацію специфічних кроків алгоритму.

Переваги: забезпечує повторне використання коду, полегшує додавання нових поведінок.

Недоліки: підкласи можуть призвести до складності коду.

Ці шаблони використовуються у великих системах, де важливі гнучкість, підтримуваність та розширюваність.

**Реалізувати не менше 3-х класів відповідно до обраної теми**

Структура проекту з реалізованими класами зображена на рисунку 1

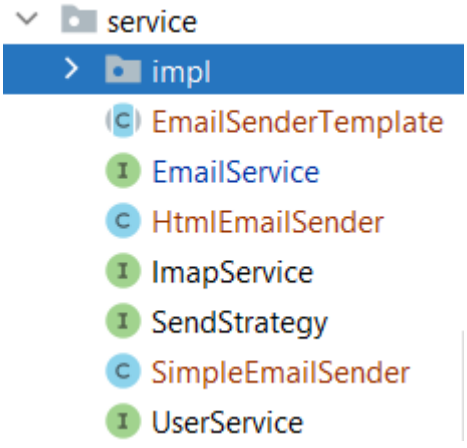


Рисунок 1 – Структура реалізованих класів

**Опис класів:**

1) Клас EmailSenderTemplate визначає загальну структуру процесу надсилання електронного листа. Він надає шаблонний метод sendEmailTemplate, що описує послідовність дій для підготовки, надсилання та обробки відповіді, залишаючи конкретну реалізацію цих кроків для підкласів через абстрактні методи.
2) Клас SimpleEmailSender є конкретною реалізацією шаблону Template Method, що визначає поведінку для підготовки, надсилання та обробки простого електронного листа, використовуючи сервіси для відправки та додаючи логування для відстеження процесу.
3) Клас HtmlEmailSender є конкретною реалізацією шаблону Template Method, що виконує підготовку, надсилання та обробку HTML-листів, використовуючи відповідний сервіс для відправки та додаючи логування для контролю процесу.

## Реалізувати один з розглянутих шаблонів за обраною темою

### Опис шаблону

Шаблон Template Method дозволяє визначити структуру виконання операції, виділивши загальну логіку в базовому класі, а змінювані частини реалізувати в підкласах. Це зменшує дублювання коду і забезпечує його гнучкість.

В реалізації Email-клієнта шаблон використовується для надсилання листів, забезпечуючи чіткий алгоритм, який включає:

```
public abstract class EmailSenderTemplate {

    public final void sendEmailTemplate(Email email) throws UnsupportedEncodingException, MessagingException {
        prepareEmail(email);
        sendEmail(email);
        handleResponse(email);
    }


    protected abstract void prepareEmail(Email email) throws UnsupportedEncodingException;

    protected abstract void sendEmail(Email email)
            throws MessagingException, UnsupportedEncodingException;

    protected abstract void handleResponse(Email email);
}
```

Рисунок 2 – клас EmailSenderTemplate

Клас EmailSenderTemplate реалізує шаблонний метод (Template Method), який задає загальну послідовність дій для відправки електронного листа. Це паттерн проєктування, який дозволяє визначити "скелет" алгоритму у вигляді незмінного методу, тоді як конкретні кроки алгоритму делегуються підкласам, що можуть забезпечувати свою специфічну реалізацію цих кроків.

Ось опис класу EmailSenderTemplate у контексті шаблону Template Method:

1. **Шаблонний метод:**

   Метод sendEmailTemplate(Email email) є основним методом, який визначає незмінну послідовність дій для відправки електронного листа. Він складається з викликів трьох основних етапів: підготовки, відправки і обробки відповіді.

2. **Абстрактні методи:**

   Методи prepareEmail, sendEmail, і handleResponse є абстрактними. Вони визначають кроки алгоритму, які повинні бути реалізовані у підкласах.

   Це дозволяє підкласам забезпечити специфічну реалізацію кожного з кроків, наприклад, підготовку спеціальних полів листа, налаштування сервера чи форматування відповіді.

```java
public class SimpleEmailSender extends EmailSenderTemplate {
    1 usage
    private final EmailService emailService;

    1 usage
    @Override
    protected void prepareEmail(Email email) throws UnsupportedEncodingException {
        System.out.println("Preparing simple email:");
        System.out.println("Subject: " + email.getSubject());
        System.out.println("Body: " + email.getBody());
    }

    1 usage
    @Override
    protected void sendEmail(Email email) throws MessagingException, UnsupportedEncodingException {
        emailService.sendEmail(email);
        System.out.println("Sending simple email to: " + email.getRecipientEmail());
    }

    1 usage
    @Override
    protected void handleResponse(Email email) {
        System.out.println("Email sent successfully at: " + email.getSentAt());
    }
}
```

Рисунок 3 – клас SimpleEmailSender

Клас SimpleEmailSender є конкретною реалізацією шаблонного класу EmailSenderTemplate, яка визначає, як саме виконуються окремі кроки алгоритму, заданого в шаблонному методі sendEmailTemplate. У контексті патерну Template Method цей клас реалізує специфічну логіку для відправки "простого" електронного листа.

1. Наслідування від базового класу:

   Клас SimpleEmailSender успадковує базовий клас EmailSenderTemplate, що забезпечує структуру алгоритму через незмінний метод sendEmailTemplate.

2. Реалізація абстрактних методів:

   prepareEmail(Email email) — реалізує підготовку електронного листа, зокрема, виводить у консоль інформацію про тему та текст повідомлення.

   sendEmail(Email email) — викликає метод emailService.sendEmail(email) для фактичної відправки листа через сервіс, а також логує одержувача.

   handleResponse(Email email) — обробляє результат відправки, наприклад, виводить час успішного відправлення у консоль.

```java
public class HtmlEmailSender extends EmailSenderTemplate {

    private final EmailService emailService;


    @Override
    protected void prepareEmail(Email email) throws UnsupportedEncodingException {
        System.out.println("Preparing HTML email:");
        System.out.println("Subject: " + email.getSubject());
        System.out.println("HTML Body: " + email.getBody());
    }


    @Override
    protected void sendEmail(Email email) throws MessagingException, UnsupportedEncodingException {
        emailService.sendEmail(email);
        email.setSentAt(LocalDateTime.now());
        System.out.println("Sending HTML email to: " + email.getRecipientEmail());
    }


    @Override
    protected void handleResponse(Email email) {
        System.out.println("HTML Email sent successfully at: " + email.getSentAt());
    }
}
```

Рисунок 4 – HtmlEmailSender

Клас HtmlEmailSender є конкретною реалізацією шаблонного класу EmailSenderTemplate, яка адаптує алгоритм відправки електронного листа для роботи з HTML-повідомленнями. У контексті патерну Template Method цей клас забезпечує специфічну логіку для відправки HTML-листів, зберігаючи загальну структуру алгоритму.

Опис класу HtmlEmailSender у контексті Template Method:

1. Наслідування базової структури:

   Клас HtmlEmailSender наслідує EmailSenderTemplate, який задає незмінну послідовність дій через метод sendEmailTemplate.

   Клас реалізує абстрактні методи базового класу для адаптації до специфіки HTML-листів.

2. Реалізація етапів алгоритму:

   - prepareEmail(Email email):

     Виконує підготовку HTML-листа, наприклад, виводячи в консоль інформацію про тему і HTML-вміст листа. Це підкреслює форматування повідомлення.

- sendEmail(Email email):

  Використовує EmailService для фактичного відправлення HTML-листа.

  Встановлює час відправки, використовуючи LocalDateTime.now(), і додає логування.

- handleResponse(Email email):

  Обробляє результат, зокрема виводить у консоль підтвердження успішного відправлення разом із часом.

**Тестування патерну**

```
@GetMapping
public ResponseEntity<Void> test() {
    try {
        Email simpleEmail = new Email();
        simpleEmail.setSubject("Meeting Reminder");
        simpleEmail.setBody("Don't forget about our meeting tomorrow at 10 AM.");
        simpleEmail.setSenderEmail("artamonovdima355@gmail.com");
        simpleEmail.setRecipientEmail("qjsoq@ukr.net");

        simpleEmailSender.sendEmailTemplate(simpleEmail);

        Email htmlEmail = new Email();
        htmlEmail.setSubject("Weekly Newsletter");
        htmlEmail.setBody("<h1>Welcome to our newsletter!</h1>");
        htmlEmail.setSenderEmail("artamonovdima355@gmail.com");
        htmlEmail.setRecipientEmail("qjsoq@ukr.net");


        htmlEmailSender.sendEmailTemplate(htmlEmail);

    } catch (UnsupportedEncodingException | MessagingException e) {
        e.printStackTrace();
    }
    return ResponseEntity.ok().build();
}
```

Рисунок 5 - тестовий метод

```
Preparing simple email:
Subject: Meeting Reminder
Body: Don't forget about our meeting tomorrow at 10 AM.

Sending simple email to: qjsoq@ukr.net
Email sent successfully at: 2024-12-20T17:08:11.083561400
Preparing HTML email:
Subject: Weekly Newsletter
HTML Body: <h1>Welcome to our newsletter!</h1>
Sending HTML email to: qjsoq@ukr.net
HTML Email sent successfully at: 2024-12-20T17:08:21.858077600
```

Рисунок 6 – результат виконання тесту

Переваги використання Template Method у поштовому клієнті

- Уніфікація логіки: Загальний алгоритм роботи з електронними листами визначений у базовому класі, що дозволяє уникнути дублювання коду для різних типів листів (текстових, HTML, з вкладеннями тощо).
- Гнучкість: Додаючи новий тип листів, достатньо реалізувати лише специфічні частини (підготовку, відправлення або обробку відповіді), не змінюючи базовий алгоритм.
- Легкість тестування: Загальна логіка (наприклад, виклик основних етапів відправки листа) тестується один раз, а специфічні деталі перевіряються окремо для кожного типу листа.
- Простота підтримки: Чітке розділення відповідальностей між базовим класом і підкласами спрощує додавання нових функцій або зміну існуючих.

Проблема, яку вирішує Template Method у поштовому клієнті

Без використання шаблону кожен тип електронного листа (текстовий, HTML, з вкладеннями тощо) мав би власну повну реалізацію алгоритму відправки. Це призвело б до дублювання коду та ускладнення підтримки через змішування загальних і специфічних частин.

Template Method вирішує цю проблему, зосереджуючи спільні кроки (наприклад, виклик сервісу відправки або логування результатів) у базовому класі, а специфічні деталі (форматування тіла листа, підготовка вкладень тощо) делегує підкласам. Це робить систему більш

масштабованою, спрощує підтримку і дає змогу легко додавати нові типи листів.

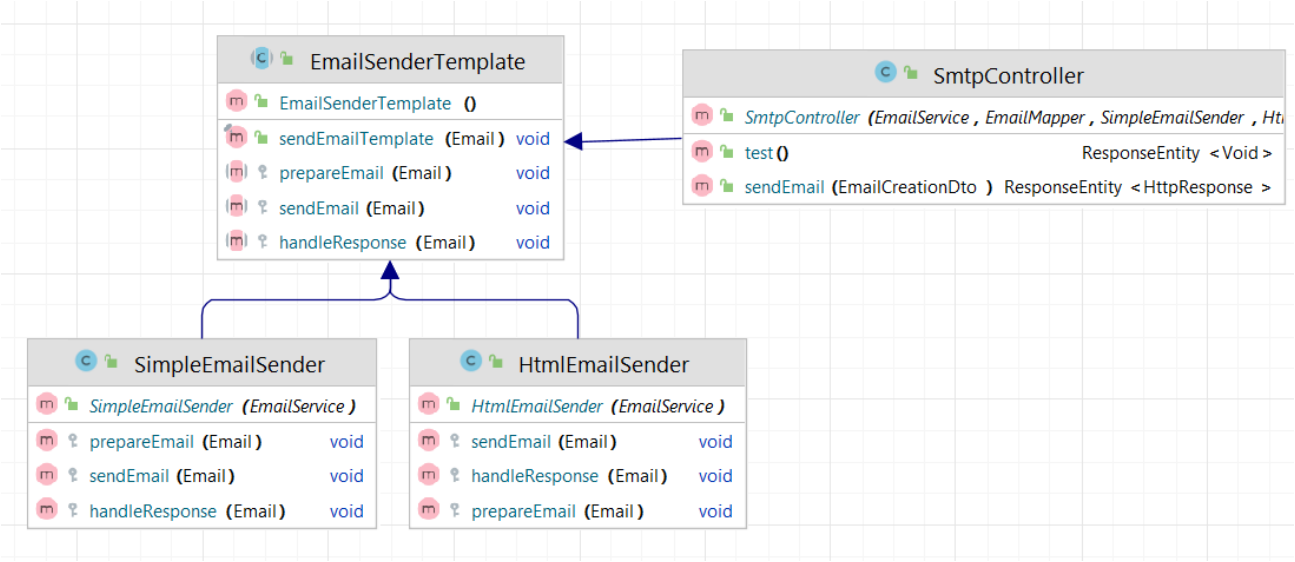



Рисунок 7 – діаграма класів

**EmailSenderTemplate**

Це базовий абстрактний клас, який визначає загальний процес роботи з електронними листами, включаючи їх підготовку, відправку та обробку відповідей. Він задає шаблон поведінки, який можуть перевизначати класи-нащадки для реалізації специфічних функцій.

**SimpleEmailSender**

Клас реалізує логіку для відправки простих текстових електронних листів. Він наслідує шаблон, визначений у базовому класі, і адаптує його для текстового формату. Цей клас виконує підготовку листа, його відправку та обробку відповіді після доставки.

**HtmlEmailSender**

Цей клас орієнтований на відправку електронних листів у форматі HTML. Використовуючи базову структуру шаблону, він додає специфіку роботи з HTML-контентом, включаючи форматування листа, його відправку та подальшу обробку.

**SmtpController**

Контролер служить посередником між клієнтськими запитами та сервісами відправки електронних листів. Він відповідає за тестування

функціональності системи та обробку запитів на відправку листів, використовуючи залежності для різних типів відправників.

Посилання на репозиторій проекту: https://github.com/qjsoq/Email-App

**Висновок**: У ході виконання лабораторної роботи було реалізовано шаблон проектування Template Method. Це дозволило структурувати виконання команд, виділити загальні етапи у базовому класі та спростити додавання нових команд через створення підкласів.

Реалізований підхід продемонстрував переваги уніфікації, зменшення дублювання коду та підвищення масштабованості системи. Використання шаблону вирішило проблему організації складних команд, що підвищило читабельність і легкість підтримки коду.